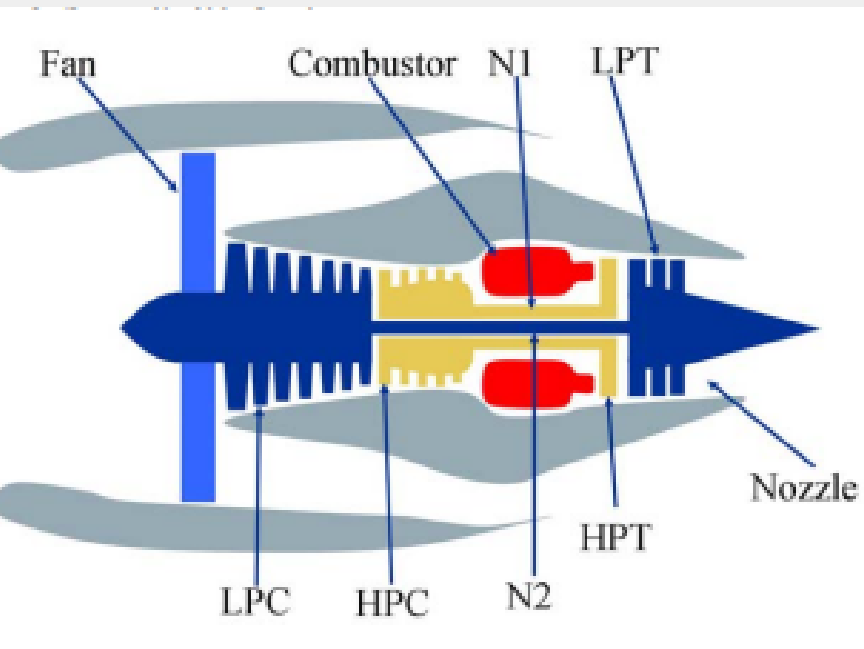


Predicting Remaining Useful Life (RUL) of Turbofan Jet Engines

Presented by:
Karam Hasan 100064840
Lara Al Omari 100064797

Presented to:
Prof. Panos Liatsis
Dr. Ibrar Amin
Dr. Kunlin Yang

Introduction & Problem Statement



◆ Background

- Predictive maintenance helps prevent unexpected failures
- Critical in systems like aircraft engines
- Improves safety, efficiency, and cost

◆ Problem Statement

- Goal: Predict Remaining Useful Life (RUL) of turbofan engines
- RUL = number of cycles before engine failure
- This is a regression problem

◆ Key Challenge

- Engines degrade over time with complex patterns
- Need to learn from sensor + operational data

Dataset Overview

◆ **Dataset Used**

- NASA C-MAPSS Turbofan Engine Dataset

◆ **Data Structure**

- 4 subsets: FD001, FD002, FD003, FD004
- Each contains:
 - Training data (run-to-failure)
 - Test data (truncated before failure)
 - True RUL labels

◆ **Key Characteristics**

- Multivariate time-series data
- Each engine has:
 - Operational settings
 - Multiple sensor readings
- Each row represents 1 engine at a specific cycle

Raw training shape: (20631, 26)
Raw test shape: (13096, 26)
Raw RUL shape: (100, 1)

	0	1	2	3	4	5	6	7	8	9	...	16	17	
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	521.66	2388.02	8138.
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	522.28	2388.07	8131.
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	522.42	2388.03	8133.
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	522.86	2388.08	8133.
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	522.19	2388.04	8133.

5 rows × 26 columns

]:

	unit_nr	cycle	op_setting_1	op_setting_2	op_setting_3	sensor_1	sensor_2	sensor_3	sensor_4
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22

5 rows × 26 columns

Total columns: 26

['unit_nr',
'cycle',
'op_setting_1',
'op_setting_2',
'op_setting_3',
'sensor_1',
'sensor_2',
'sensor_3',
'sensor_4',
'sensor_5',
'sensor_6',
'sensor_7',
'sensor_8',
'sensor_9',
'sensor_10',
'sensor_11',
'sensor_12',
'sensor_13',
'sensor_14',
'sensor_15',
'sensor_16',
'sensor_17',
'sensor_18',
'sensor_19',
'sensor_20',
'sensor_21']

Data Exploration

- ◆ Initial Analysis

- Checked data types and structure
- Verified dataset integrity

- ◆ Key Findings

- No missing values
- 26 features per observation
- Mostly numerical data

- ◆ Observations

- Some features have very low or zero variance
- Potential need for feature selection

```
9]: # check data types and non-null counts
train_fd001.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20631 entries, 0 to 20630
Data columns (total 26 columns):
#   Column      Non-Null Count  Dtype
---  -
0   unit_nr      20631 non-null  int64
1   cycle        20631 non-null  int64
2   op_setting_1 20631 non-null  float64
3   op_setting_2 20631 non-null  float64
4   op_setting_3 20631 non-null  float64
5   sensor_1     20631 non-null  float64
6   sensor_2     20631 non-null  float64
7   sensor_3     20631 non-null  float64
8   sensor_4     20631 non-null  float64
9   sensor_5     20631 non-null  float64
10  sensor_6     20631 non-null  float64
11  sensor_7     20631 non-null  float64
12  sensor_8     20631 non-null  float64
13  sensor_9     20631 non-null  float64
14  sensor_10    20631 non-null  float64
15  sensor_11    20631 non-null  float64
16  sensor_12    20631 non-null  float64
17  sensor_13    20631 non-null  float64
18  sensor_14    20631 non-null  float64
19  sensor_15    20631 non-null  float64
20  sensor_16    20631 non-null  float64
21  sensor_17    20631 non-null  int64
22  sensor_18    20631 non-null  int64
23  sensor_19    20631 non-null  float64
24  sensor_20    20631 non-null  float64
25  sensor_21    20631 non-null  float64
dtypes: float64(22), int64(4)
memory usage: 4.1 MB
```

	count	mean	std	min	25%	50%	75%	
unit_nr	20631.0	51.506568	2.922763e+01	1.0000	26.0000	52.0000	77.0000	100
cycle	20631.0	108.807862	6.888099e+01	1.0000	52.0000	104.0000	156.0000	362
op_setting_1	20631.0	-0.000009	2.187313e-03	-0.0087	-0.0015	0.0000	0.0015	0
op_setting_2	20631.0	0.000002	2.930621e-04	-0.0006	-0.0002	0.0000	0.0003	0
op_setting_3	20631.0	100.000000	0.000000e+00	100.0000	100.0000	100.0000	100.0000	100
sensor_1	20631.0	518.670000	0.000000e+00	518.6700	518.6700	518.6700	518.6700	518
sensor_2	20631.0	642.680934	5.000533e-01	641.2100	642.3250	642.6400	643.0000	644
sensor_3	20631.0	1590.523119	6.131150e+00	1571.0400	1586.2600	1590.1000	1594.3800	1616
sensor_4	20631.0	1408.933782	9.000605e+00	1382.2500	1402.3600	1408.0400	1414.5550	1441
sensor_5	20631.0	14.620000	1.776400e-15	14.6200	14.6200	14.6200	14.6200	14
sensor_6	20631.0	21.609803	1.388985e-03	21.6000	21.6100	21.6100	21.6100	21
sensor_7	20631.0	553.367711	8.850923e-01	549.8500	552.8100	553.4400	554.0100	556
sensor_8	20631.0	2388.096652	7.098548e-02	2387.9000	2388.0500	2388.0900	2388.1400	2388
sensor_9	20631.0	9065.242941	2.208288e+01	9021.7300	9053.1000	9060.6600	9069.4200	9244
sensor_10	20631.0	1.300000	0.000000e+00	1.3000	1.3000	1.3000	1.3000	1
sensor_11	20631.0	47.541168	2.670874e-01	46.8500	47.3500	47.5100	47.7000	48
sensor_12	20631.0	521.413470	7.375534e-01	518.6900	520.9600	521.4800	521.9500	523
sensor_13	20631.0	2388.096152	7.191892e-02	2387.8800	2388.0400	2388.0900	2388.1400	2388
sensor_14	20631.0	8143.752722	1.907618e+01	8099.9400	8133.2450	8140.5400	8148.3100	8293
sensor_15	20631.0	8.442146	3.750504e-02	8.3249	8.4149	8.4389	8.4656	8
sensor_16	20631.0	0.030000	1.387812e-17	0.0300	0.0300	0.0300	0.0300	0
sensor_17	20631.0	393.210654	1.548763e+00	388.0000	392.0000	393.0000	394.0000	400
sensor_18	20631.0	2388.000000	0.000000e+00	2388.0000	2388.0000	2388.0000	2388.0000	2388
sensor_19	20631.0	100.000000	0.000000e+00	100.0000	100.0000	100.0000	100.0000	100
sensor_20	20631.0	38.816271	1.807464e-01	38.1400	38.7000	38.8300	38.9500	39
sensor_21	20631.0	23.289705	1.082509e-01	22.8942	23.2218	23.2979	23.3668	23

Engine Degradation Over Time

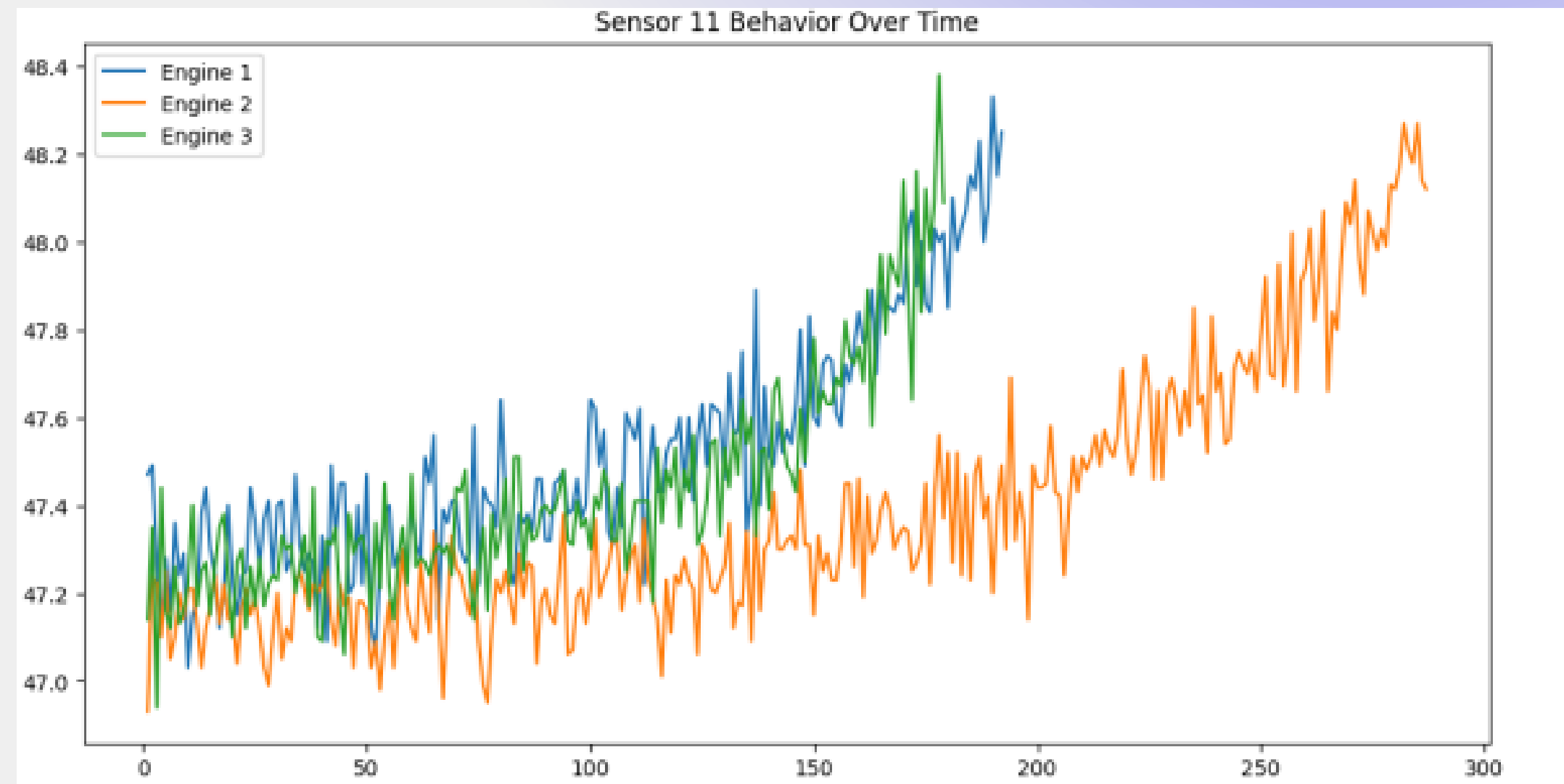
04

- ◆ **Visualization**

- Sensor values plotted across cycles
- Multiple engines compared

- ◆ **Observations**

- Sensor values change over time
- Clear degradation trends
- Some sensors strongly reflect engine health



Target Variable: Remaining Useful Life (RUL)

◆ Definition

- RUL = Remaining Useful Life
- Number of cycles before engine failure

◆ How it is computed

- For training data:
 - $RUL = \text{max cycle of engine} - \text{current cycle}$

◆ Key Properties

- RUL decreases over time
- Final cycle $\rightarrow RUL = 0$

	unit_nr	cycle	max_cycle	RUL
0	1	1	192	191
1	1	2	192	190
2	1	3	192	189
3	1	4	192	188
4	1	5	192	187
5	1	6	192	186
6	1	7	192	185
7	1	8	192	184
8	1	9	192	183
9	1	10	192	182

	cycle	RUL
182	183	9
183	184	8
184	185	7
185	186	6
186	187	5
187	188	4
188	189	3
189	190	2
190	191	1
191	192	0

RUL Capping

06

◆ Problem with Raw RUL

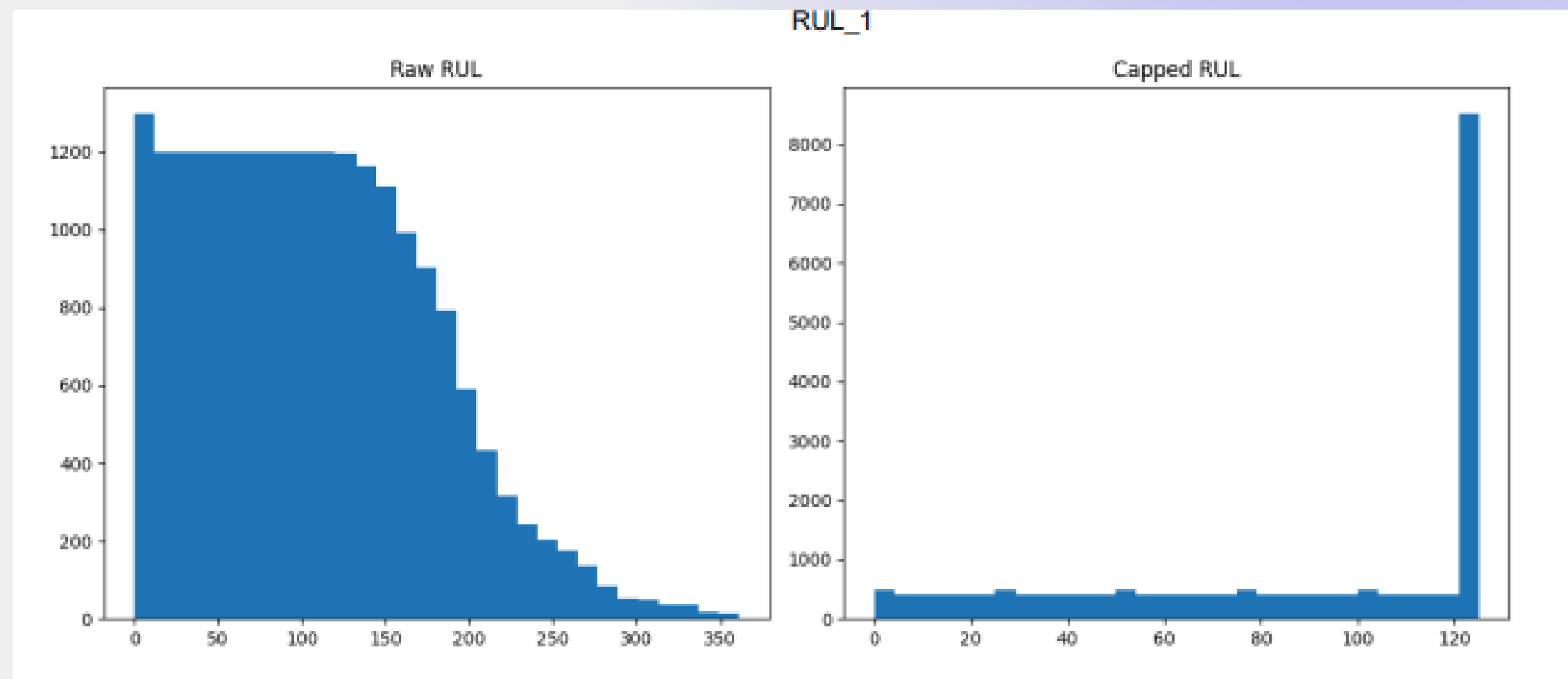
- Many samples have very high RUL values
- Early engine life dominates the dataset

◆ Solution

- Apply a cap:
 - $RUL = \min(RUL, 125)$

◆ Effect

- Reduces impact of early-life samples
- Focuses on degradation region (near failure)
- Improves model learning



Feature Engineering

◆ Problem

- Some features have no variation
- Do not contribute to prediction

◆ Solution

- Removed constant features

◆ Result

- Reduced noise
- Improved model efficiency
- Simplified dataset

```
22]: # check number of unique values per  
train_fd001.nunique().sort_values()
```

```
22]: sensor_1      1  
      op_setting_3 1  
      sensor_10    1  
      sensor_5     1  
      sensor_19    1  
      sensor_16    1  
      sensor_18    1  
      sensor_6     2  
      sensor_17    13  
      op_setting_2 13  
      sensor_8     53  
      sensor_13    56  
      unit_nr      100  
      sensor_20    120  
      RUL          126  
      op_setting_1 158  
      sensor_11    159  
      sensor_2     310  
      cycle        362  
      sensor_12    427  
      sensor_7     513  
      sensor_15    1918  
      sensor_3     3012  
      sensor_4     4051  
      sensor_21    4745  
      sensor_14    6078  
      sensor_9     6403  
      dtype: int64
```

```
# find columns with only one unique value
```

```
constant_cols = [col for col in train_fd001.columns if train_fd001[col].nunique() == 1]  
  
print("Constant columns:")  
constant_cols
```

```
Constant columns:
```

```
['op_setting_3',  
 'sensor_1',  
 'sensor_5',  
 'sensor_10',  
 'sensor_16',  
 'sensor_18',  
 'sensor_19']
```

Feature Importance & Correlation

08

◆ Correlation Analysis

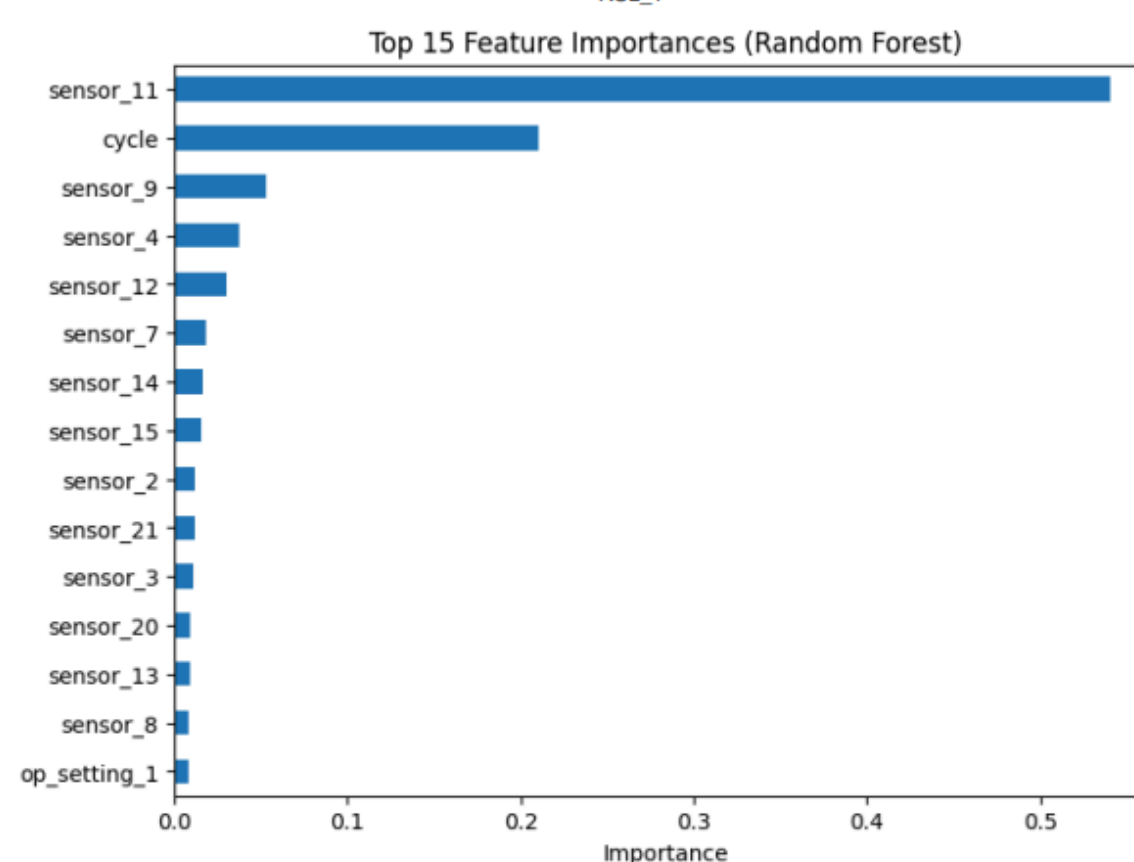
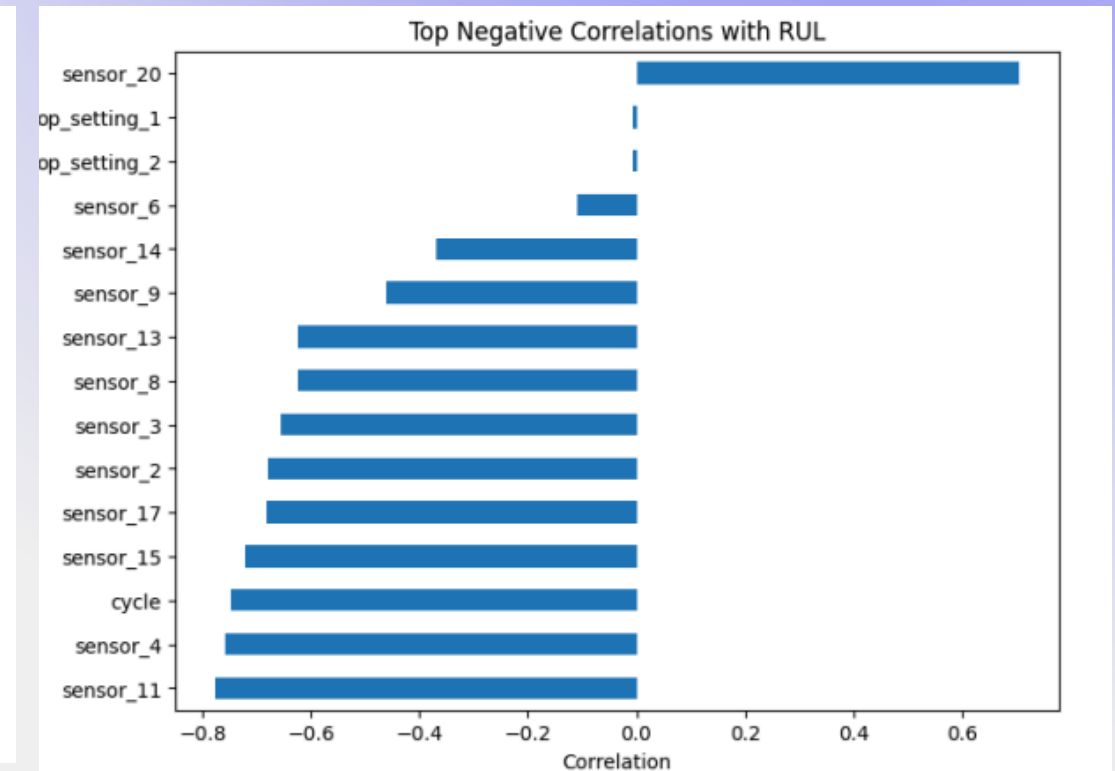
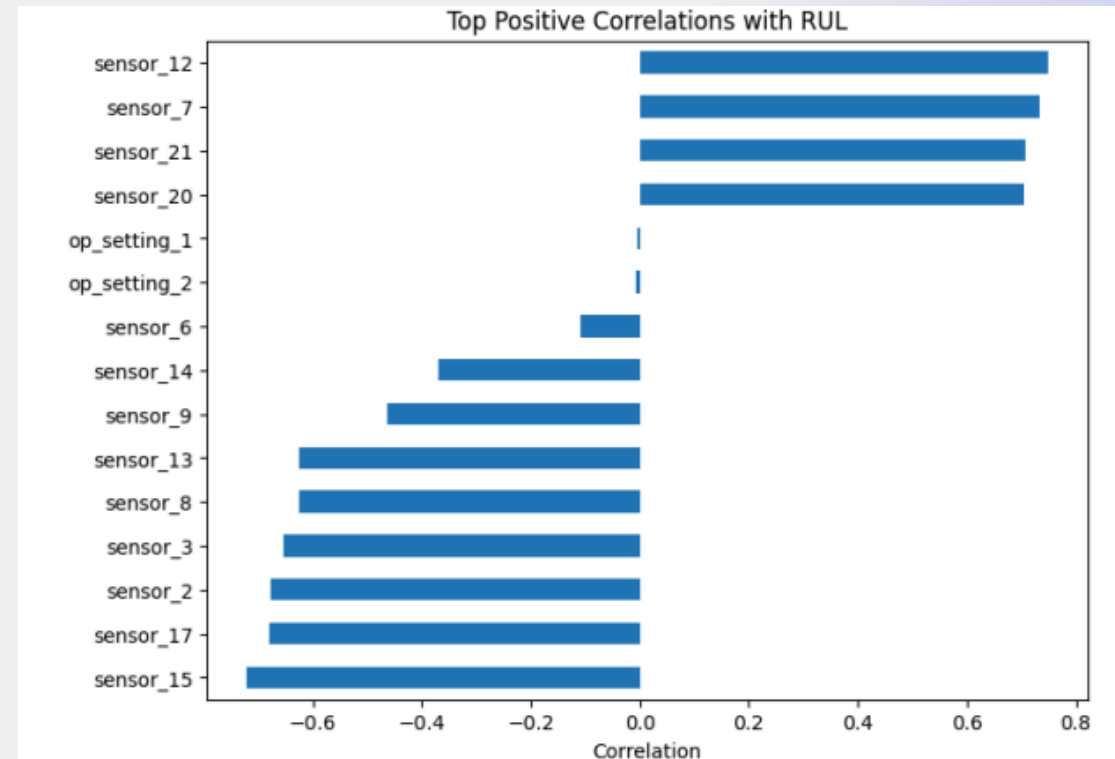
- Measured relationship between features and RUL
- Identified strong predictors of degradation

◆ Key Findings

- Strong negative correlation:
 - sensor_11, sensor_4, cycle
- Strong positive correlation:
 - sensor_12, sensor_7

Feature Importance (Random Forest)

- Most important feature:
 - sensor_11
- Followed by:
 - cycle, sensor_9, sensor_4



```
selected_features = feature_importance.h  
selected_features
```

```
[ 'sensor_11',  
  'cycle',  
  'sensor_9',  
  'sensor_4',  
  'sensor_12',  
  'sensor_7',  
  'sensor_14',  
  'sensor_15',  
  'sensor_2',  
  'sensor_21' ]
```

Train/Validation Split

09

◆ Approach

- Split data at engine level (not rows)

◆ Why?

- Prevents data leakage
- Ensures realistic evaluation

Split Ratio

- 80% training engines
- 20% validation engines

```
train_ids, val_ids = train_test_split(engine_ids, test_size=0.2,  
  
print("Train engines:", len(train_ids))  
print("Validation engines:", len(val_ids))
```

```
Train engines: 80  
Validation engines: 20
```

```
: # create train and validation datasets  
train_df = train_fd001[train_fd001['unit_nr'].isin(train_ids)]  
val_df = train_fd001[train_fd001['unit_nr'].isin(val_ids)]  
  
print("Train shape:", train_df.shape)  
print("Validation shape:", val_df.shape)
```

```
Train shape: (16561, 20)  
Validation shape: (4070, 20)
```

Models Used

Traditional Machine Learning Models

- **Baseline Models**

- Linear Regression
- Ridge

- **Distance Models**

- KNN
- SVR

- **Tree-Based Models**

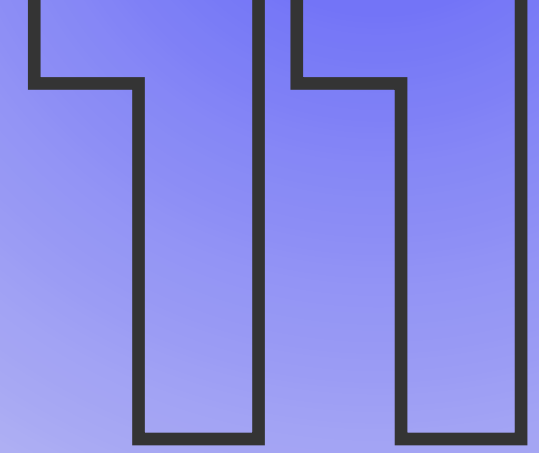
- Decision Tree
- Random Forest
- XGBoost

```
models = {  
    "Linear": LinearRegression(),  
    "Ridge": Ridge(alpha=1.0),  
    "KNN": KNeighborsRegressor(n_neighbors=5),  
    "SVR": SVR(kernel='rbf', C=10, epsilon=0.1),  
    "Decision Tree": DecisionTreeRegressor(max_depth=10, random_state=42),  
    "Random Forest": RandomForestRegressor(n_estimators=300, max_depth=20, min_  
                                           min_samples_leaf=2, random_state=42),  
    "XGBoost": XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05, s  
                             objective='reg:squarederror', random_state=42  
)
```

- ◆ **Why Multiple Models?**

- Compare different learning behaviors
- Identify the best-performing approach

Evaluation Metrics



◆ Metrics Used

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- R^2 Score (Coefficient of Determination)

◆ What They Measure

- MAE → Average prediction error
- RMSE → Penalizes large errors
- R^2 → How well the model explains variance

We train each model on the training set and evaluate it on the validation set using:

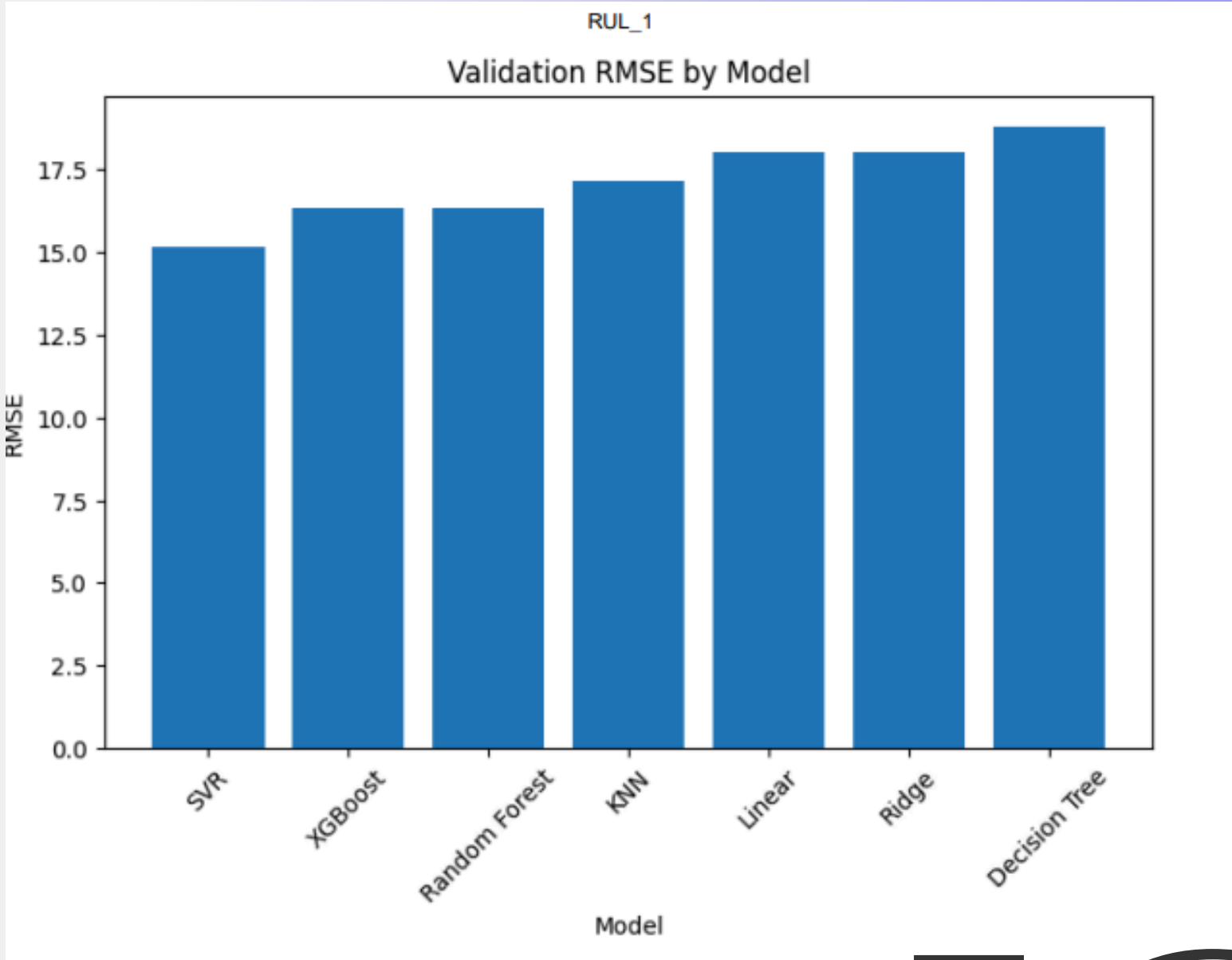
- MAE
- RMSE
- R^2 score

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Model Performance (FD001)

]:

	Model	MAE	RMSE	R2
3	SVR	9.909042	15.179382	0.867593
6	XGBoost	10.987623	16.326437	0.846826
5	Random Forest	11.042743	16.334747	0.846670
2	KNN	11.265946	17.158024	0.830825
0	Linear	14.597545	18.024448	0.813308
1	Ridge	14.597662	18.024585	0.813305
4	Decision Tree	12.517163	18.793393	0.797039



Cross-Validation Results

13

◆ Why Cross-Validation?

- Avoid results depending on a single split
- Ensure model stability

◆ Models Evaluated

- SVR
- Random Forest
- XGBoost

	Model	Avg R2	Std R2	Avg RMSE	Std RMSE
0	SVR	0.829686	0.025473	17.140651	1.281060
1	Random Forest	0.828787	0.021363	17.202578	1.009389
2	XGBoost	0.827184	0.018607	17.290681	0.887315

Hyperparameter Tuning (Optuna)

14

◆ Objective

- Improve model performance
- Find optimal parameters

◆ Method Used

- Optuna (automated hyperparameter tuning)
- Searches for best parameter combinations

◆ Tuned Parameters (Random Forest)

- Number of trees (n_estimators)
- Tree depth (max_depth)
- Min samples split / leaf
- Feature selection (max_features)

```
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)
```

```
print("Best trial:")
print("Best R2:", study.best_trial.value)
print("Best parameters:")
print(study.best_trial.params)
```

```
53453.
[I 2026-04-15 12:13:57,710] Trial 26 finished with value: 0.8625071379439265 and p
arameters: {'n_estimators': 200, 'max_depth': 8, 'min_samples_split': 9, 'min_sam
ples_leaf': 2, 'max_features': 'sqrt'}. Best is trial 22 with value: 0.862747531435
3453.
[I 2026-04-15 12:14:01,986] Trial 27 finished with value: 0.8595904759304827 and p
arameters: {'n_estimators': 200, 'max_depth': 12, 'min_samples_split': 5, 'min_sam
ples_leaf': 2, 'max_features': 'sqrt'}. Best is trial 22 with value: 0.86274753143
53453.
[I 2026-04-15 12:14:05,539] Trial 28 finished with value: 0.8596697734071614 and p
arameters: {'n_estimators': 300, 'max_depth': 6, 'min_samples_split': 6, 'min_sam
ples_leaf': 1, 'max_features': 'sqrt'}. Best is trial 22 with value: 0.862747531435
3453.
[I 2026-04-15 12:14:20,045] Trial 29 finished with value: 0.847651318409377 and pa
rameters: {'n_estimators': 200, 'max_depth': 22, 'min_samples_split': 10, 'min_sam
ples_leaf': 2, 'max_features': None}. Best is trial 22 with value: 0.8627475314353
453.
```

Best trial:

Best R2: 0.8627475314353453

Best parameters:

```
{'n_estimators': 100, 'max_depth': 8, 'min_samples_split': 7, 'min_samples_leaf':
2, 'max_features': 'sqrt'}
```

Final Model Results (Random Forest)

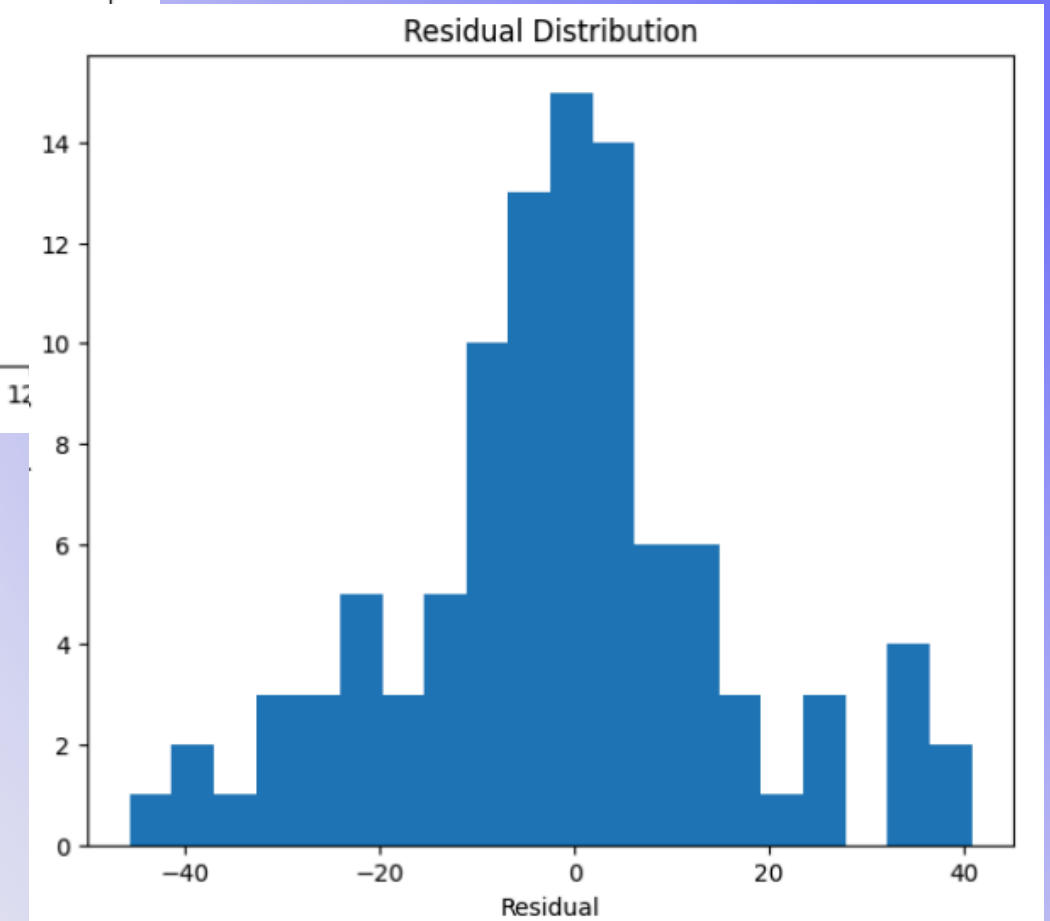
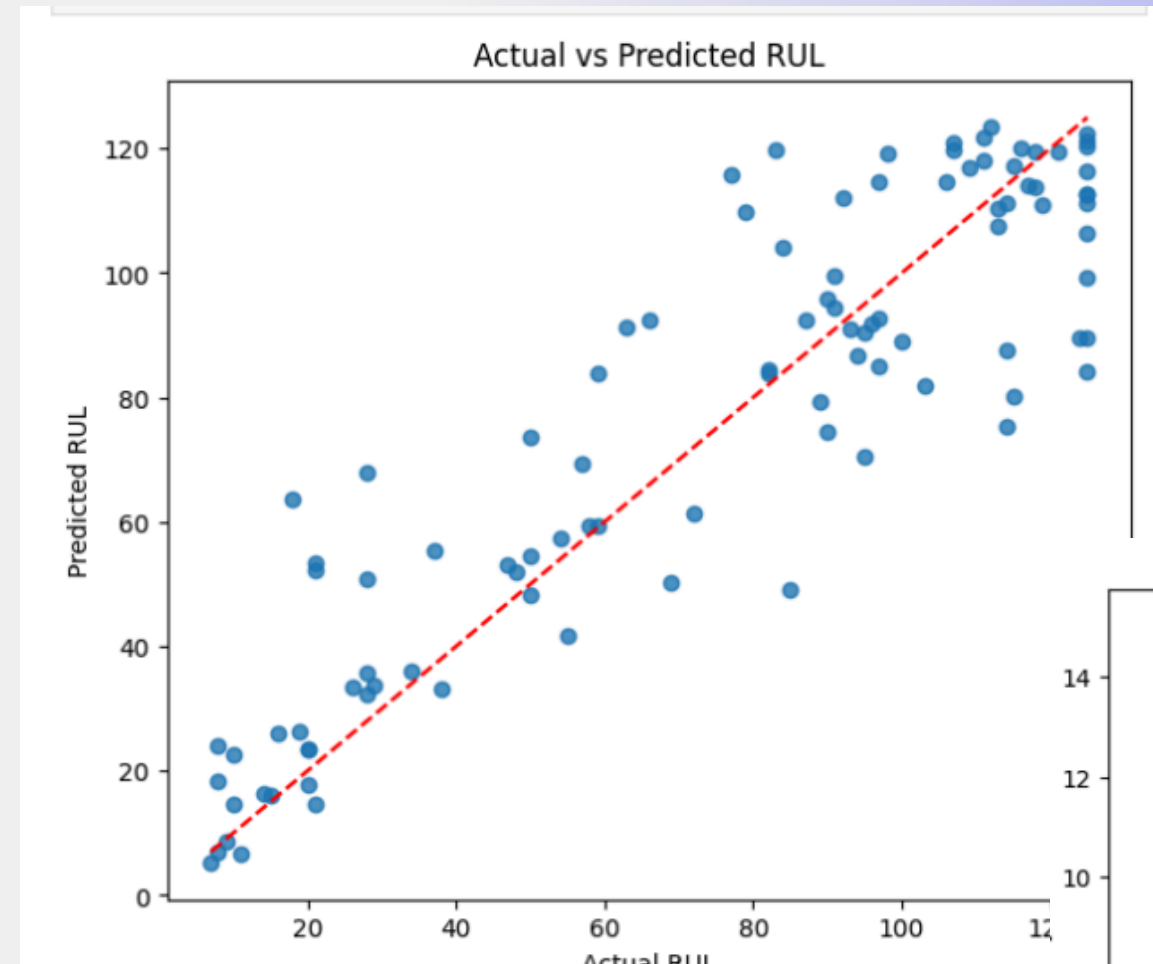
15

◆ Final Performance (FD001 Test Set)

- MAE ≈ 12.7
- RMSE ≈ 17.25
- $R^2 \approx 0.81$

◆ Key Observations

- Good overall prediction accuracy
- Captures degradation trend
- Some errors at high RUL values



DeepLearning Approach: 1D CNN

16

◆ Motivation

- Traditional models treat data as independent rows
- But this is time-series data

◆ Key Idea

- Use sequences of sensor readings
- Capture temporal degradation patterns

◆ Why CNN?

- Learns patterns across time
- Detects local trends in sequences
- Better suited for time-series data

```
def create_train_sequences(df, feature_cols, seq_len):  
    X, y = [], []  
  
    for engine_id in df['unit_nr'].unique():  
        engine_data = df[df['unit_nr'] == engine_id].sort_values('cycle')  
  
        features = engine_data[feature_cols].values  
        targets = engine_data['RUL'].values  
  
        for i in range(len(engine_data) - seq_len + 1):  
            X.append(features[i:i+seq_len])  
            y.append(targets[i+seq_len-1])  
  
    return np.array(X, dtype=np.float32), np.array(y, dtype=np.float32)
```

CNN Validation Performance (Before Tuning)

◆ Validation Performance

- MAE ≈ 9.9
- RMSE ≈ 13.4
- $R^2 \approx 0.898$

◆ Comparison with ML Models

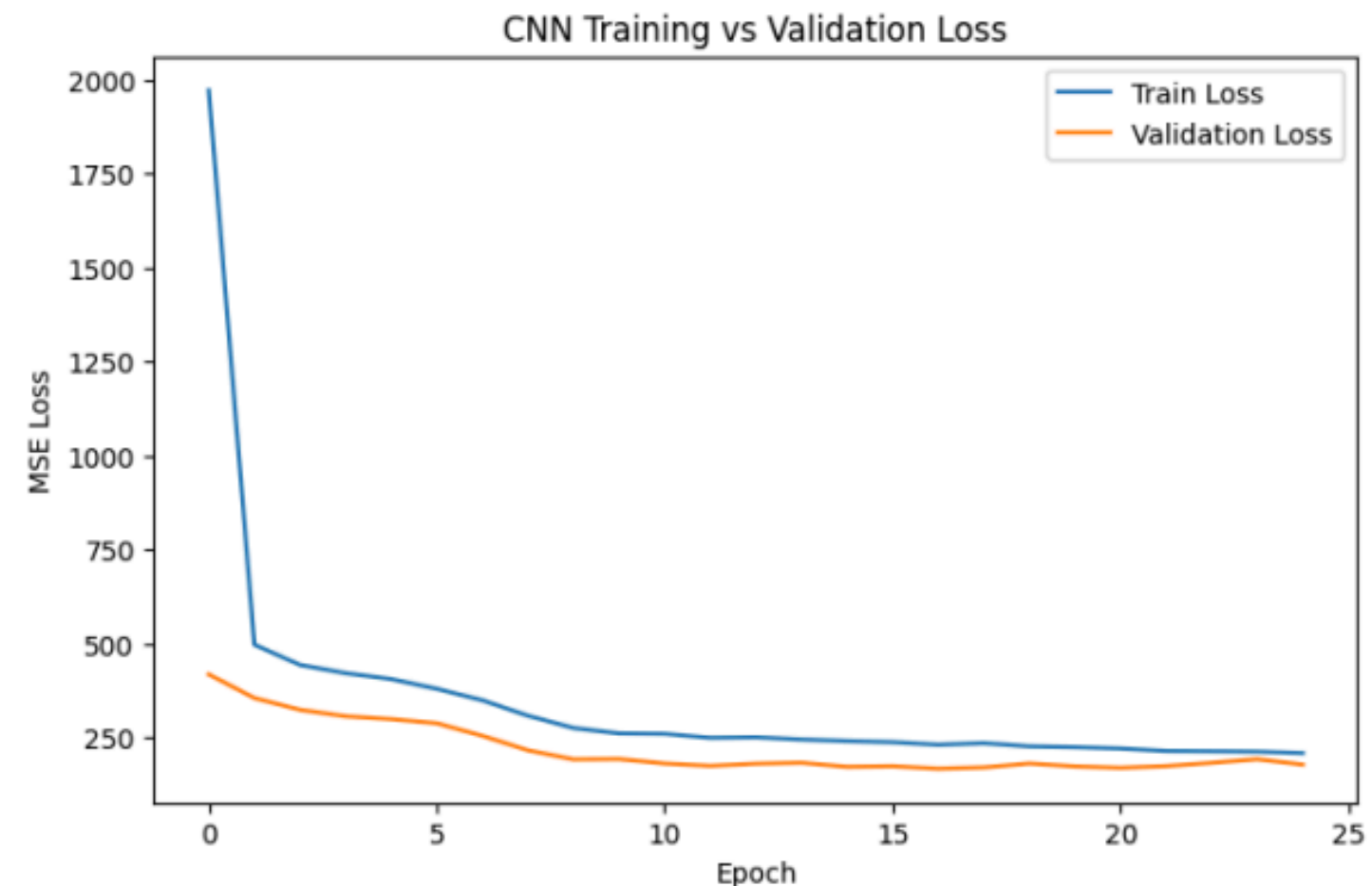
- CNN outperforms Random Forest
- Better at capturing degradation patterns

◆ Key Observations

- Strong prediction accuracy
- Captures temporal behavior
- Slight errors at high RUL

CNN Validation Results:
MAE: 9.87984848022461
RMSE: 13.394794398115017
R2: 0.8970840573310852

```
plt.figure(figsize=(8,5))
plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("CNN Training vs Validation Loss")
plt.legend()
plt.show()
```



CNN Hyperparameter Optimization (Optuna)

18

◆ Objective

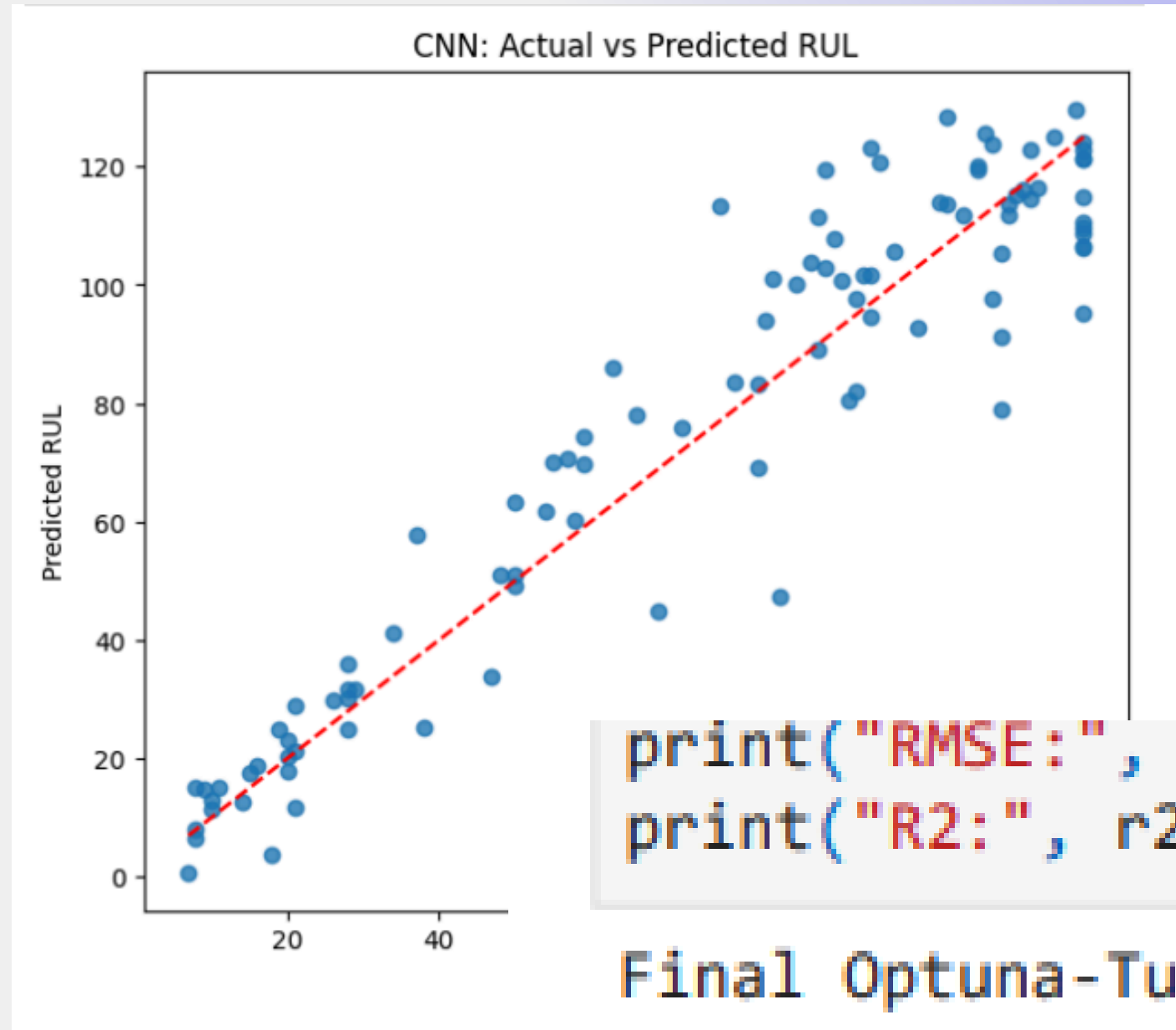
- Improve CNN performance
- Find optimal parameters

◆ Tuned Parameters

- Number of filters
- Hidden layer size
- Learning rate
- Dropout
- Batch size

◆ Final Results

- Model didn't improve much after tuning
- Test Set Results were really good



```
print("RMSE:", rmse_cnn)  
print("R2:", r2_cnn)
```

Final Optuna-Tuned CNN Results:

MAE: 9.675686836242676

RMSE: 12.889485034821625

R2: 0.8965426087379456

```
cnn_residuals = y_official_test -
```

Scalable Modular Pipeline

19

◆ Motivation

- Avoid code repetition
- Ensure consistency across datasets

◆ Approach

- Created reusable functions for:
 - Data loading
 - RUL computation
 - Feature selection
 - Model training & evaluation

◆ Key Idea

- Same pipeline → applied to FD002, FD003, FD004

```
] : %reload_ext autoreload  
    %autoreload 2
```

am/Downloads/RUL_1.html

RUL_

```
from rul_functions import *
```

```
] : import rul_functions  
    print(rul_functions.__file__)
```

C:\Users\karam\ML_PROJECT\rul_functions.py

Dataset Complexity

20

Dataset	Conditions	Fault Modes	Difficulty
FD001	1	1	EASY
FD002	MULTIPLE	1	MEDIUM
FD003	1	MULTIPLE	MEDIUM
FD004	MULTIPLE	MULTIPLE	HARD

Model Performance Across Datasets

21

DATASET	MODEL	R ²
FD001	CNN(OPTUNA)	0.90
FD002	CNN	0.69
FD003	SVR	0.76
FD004	XGBOOST	0.57

◆ Observations

- Performance decreases as complexity increases
- CNN performs best on simpler datasets
- Smaller improvement on complex datasets

CONCLUSION

22

◆ Achievements

- Built a complete ML → DL pipeline
- Compared traditional and deep learning models
- Successfully modeled engine degradation

◆ Key Insights

- Time-series modeling significantly improves RUL prediction
- Data complexity is the main limiting factor
- CNN captures degradation patterns better than static models

◆ Project Understanding

- This project was exploratory in nature
- Focus was on understanding the problem and model behavior
- Gained insight into:
 - how engines degrade
 - how models interpret temporal data

◆ Future Work

- LSTM / advanced sequence models
- Improved feature engineering
- More robust handling of complex datasets

Thank You

Questions?